

# PRISM-TopoMap C++ 重构 Bug 排查与修复总结

## 概述

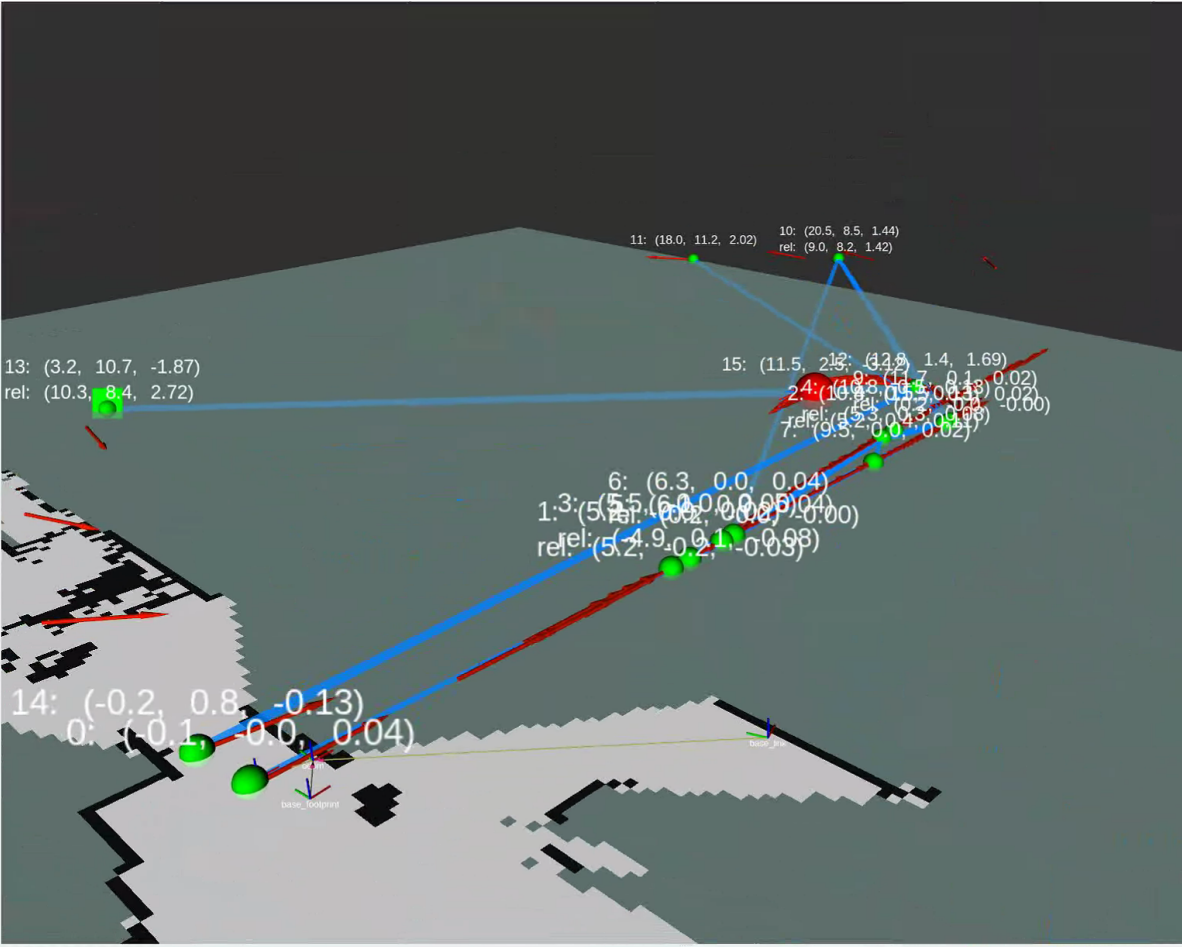
PRISM-TopoMap 是一个基于拓扑图的在线 SLAM 系统。本项目将其从纯 Python 重构为 C++/Python 混合架构。在 C++ 版本测试过程中，发现并修复了多个导致建图失败的 bug。本文档按发现顺序记录每个问题、根因分析、修复方案及验证结果。

## Bug 1: odometry 数据源错误 — 用 GT 替代 /odom 做里程计积分

### 现象

C++ 版本运行日志 (log5) 显示拓扑图出现"节点爆炸"和"坐标乱飞": `rel_pose_vcur` 的 `theta` 从 0.04 rad 瞬间漂到 -3.19 rad, `grid_shift` 达到 12.3m, 顶点坐标无序跳跃。

```
DIAG] sync.global_pose=(7.7267, 0.0324, 0.0264) stamp=1517156177.143[0m
DIAG] grid_shift=(-0.2544, 0.0021, 0.0035) rel_pose_vcur=(12.30, 0.15, -3.19)[0m
```



## 根因

[src/prism\\_topomap\\_node.cpp:342-348](#) (修复前) :

```
if (use_odom_) {  
    // 从 /odom 获取里程计  
    ...  
} else {  
    result.odom_pose = result.global_pose; // ← BUG: 用 GT 替代里程计  
}
```

配置文件 [scout\\_rosbag.yaml:14](#) 中 `use_odom: false` 触发了此路径。Python 原版 [prism\\_topomap\\_node.py:734-735](#) 始终用 `/odom` 做里程计积分, `/odom_gt` 仅做可视化。我修改时错误地将 GT 姿态当做里程计增量输入。

日志证据 — log5 line 374:

```
grid_shift=(6.2258, 0.4258, 0.0707) rel_pose_vcur=(6.18, 0.05, -3.11)
```

GT 姿态在连续处理帧间存在大幅跳变, 直接导致 `grid_shift` 和 `rel_pose_of_vcur` 计算错误。

## 修复

[src/prism\\_topomap\\_node.cpp:342-366](#) (修复后) :

```
// 始终从 /odom topic 获取里程计位姿 (匹配 Python 行为)  
{  
    double odom_diff = std::numeric_limits<double>::max();  
    if (getNearestPose(odom_poses_, result.odom_pose, odom_diff) &&  
        odom_diff <= kPoseSyncTolerance) {  
        // 成功从 /odom 获取  
    } else {  
        ROS_WARN_THROTTLE(5.0, "No odometry data, falling back to  
global_pose...");  
        result.odom_pose = result.global_pose; // 仅在 /odom 不可用时回退  
    }  
}
```

## 结果

log11 中 odometry delta 正常 (~0.0007m/frame), 不再出现 theta 漂移爆炸。

---

## Bug 2: GT topic 缺失导致系统完全卡死

### 现象

更换 rosbag 后, C++ 日志 (log10) 显示 `waiting for pose data to catch up... gt_poses=0, odom_poses=656`, 系统一帧都无法处理, RVIZ 完全空白。

## 根因

[src/prism\\_topomap\\_node.cpp:311](#) (修复前) :

```
if (use_gt_pose_) {  
    if (gt_poses_.empty()) return result; // ← 直接返回 valid=false
```

getSyncPoseAndImages 中 `gt_poses_` 为空时直接返回 `valid=false`。processPcdQueue 遇到 `!sync.valid` 后不弹出队列而是 `break`，导致第一帧永远卡在队首，形成死锁。

**确认：**`rosviz info` 显示当前 rosviz 根本不含 `/odom_gt` topic，只有 `/odom`，`/rslidar_points`，`/tf` 等。Python 原版 [prism\\_topomap\\_node.py:749-751](#) 对 `cur_global_pose is None` 只是 `print` + `return` 跳过当前帧，不阻塞后续帧。

## 修复

[src/prism\\_topomap\\_node.cpp:310-347](#) (修复后) :

```
if (use_gt_pose_ && !gt_poses_.empty()) {  
    // 从 GT 获取 global_pose  
} else {  
    // 回退：从 odometry 获取 global_pose  
    getNearestPose(odom_poses_, result.global_pose, odom_diff);  
    if (use_gt_pose_ && gt_poses_.empty()) {  
        ROS_WARN_THROTTLE(5.0, "GT topic %s has no data, falling back to  
/odom...");  
    }  
}
```

## 结果

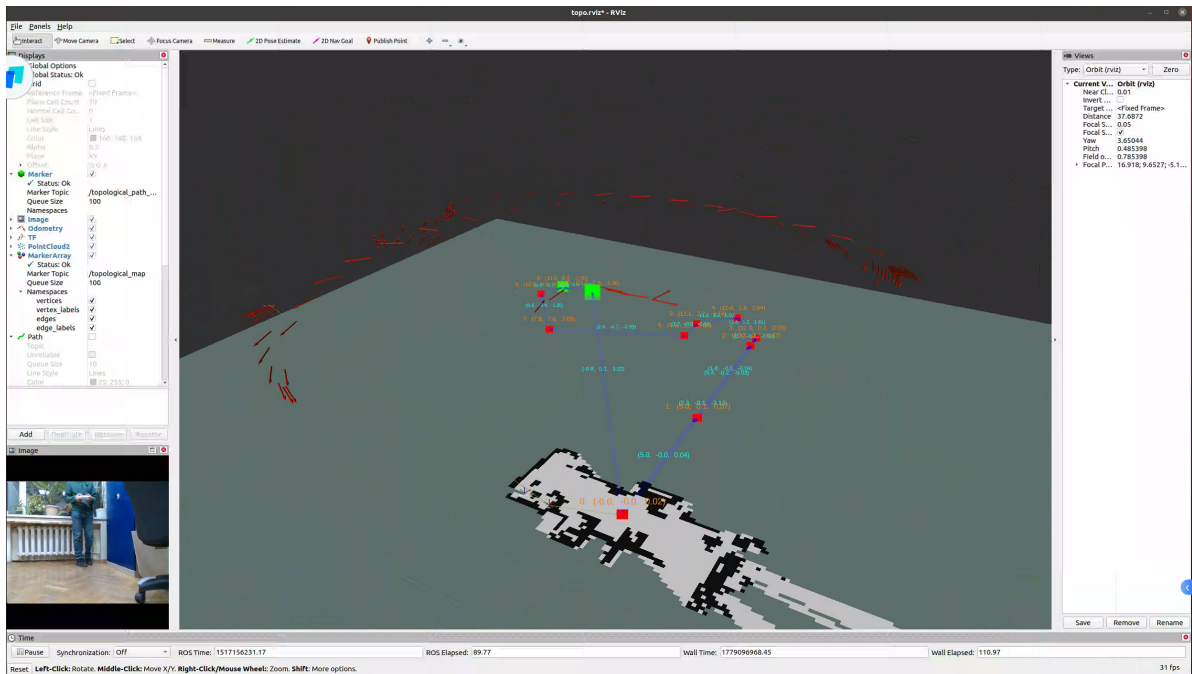
log11 系统正常运行，生成 11 个顶点，轨迹完整。

---

## Bug 3: `publishTfFromOdom` 发布错误的 TF 帧关系

### 现象

RVIZ 中 `base_footprint` 坐标系一直乱飞，和地图对不上。（就是图中的红色箭头，代表在机器人上的基坐标系的二维朝向）



## 根因

[src/results\\_publisher.cpp:395-406](#) (修复前)：

```
void ResultsPublisher::publishTfFromOdom(double x, double y, double theta, ...)
{
    tf_broadcaster_.sendTransform(
        tf::StampedTransform(transform, stamp, map_frame_, odom_frame));
    //                                     ^^^^^^^^^^^ ^^^^^^^^^^^
    //                                     硬编码 parent="map", child="odom"
}
```

Python 原版 [prism\\_topomap\\_node.py:472-480](#):

```
def publish_tf_from_odom(self, msg):
    self.tfbr.sendTransform((x, y, 0), ...,
                            msg.header.stamp,
                            msg.child_frame_id,    # ← 从消息中读取
                            msg.header.frame_id)    # ← 从消息中读取
```

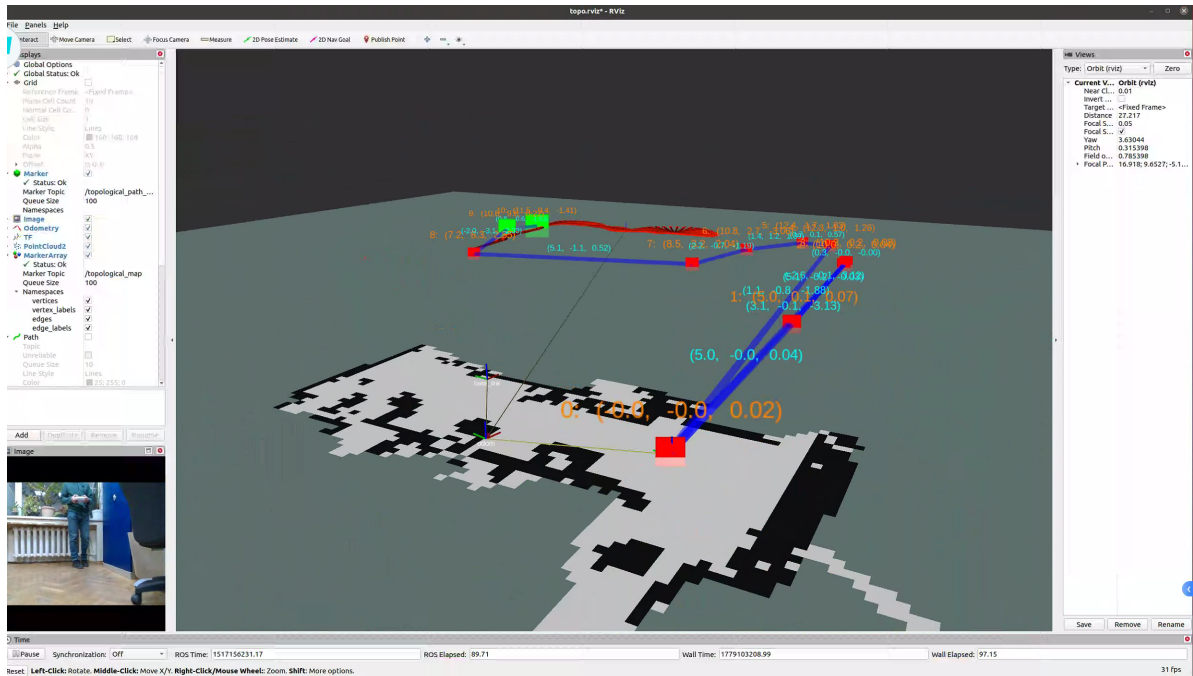
Python 使用 odometry 消息自带的 `child_frame_id` (通常是 `base_footprint`) 和 `header.frame_id` (通常是 `odom`)，发布 `odom → base_footprint`。C++ 硬编码发布 `map → odom`，用原始里程计位姿作为 `map` 到 `odom` 的变换，导致 TF 树完全错乱。

## 修复

[src/results\\_publisher.cpp:395-411](#) (修复后)



```
void ResultsPublisher::publishTfFromOdom(const nav_msgs::Odometry::ConstPtr&
msg) {
    // 与 Python 一致：使用消息中的 frame_id 和 child_frame_id
    tf_broadcaster_.sendTransform(
        tf::StampedTransform(transform, msg->header.stamp,
                               msg->header.frame_id,      // parent
                               msg->child_frame_id));       // child
}
```



## Bug 4: PCD 帧率限流过于激进 (0.5s → 0.1s → 0.3s)

### 现象

PCD就是处理连续图像帧时的帧率，太快会导致误差偏移累积快从而里程计漂移，太慢代表按照速率跳过了更多帧导致信息变少

```
: [THROTTLE] Skipping PCD frame (stamp=1517156230.639, diff=0.499s, skipped_total=737, queue=1, gt_buf=0, odom_buf=892) esc
011,0.0386,3.5704) esc[0m
```

- **0.5s:** 跳过 83% 帧，定位响应延迟（就是图中的skipped\_total,占了总odom\_buf的83%）
- **0.1s:** 瞬时 IoU 波动触发过早顶点创建，vtx2 和 vtx3 仅距 0.2m (log13)
- **0.2s:** vtx2 和 vtx3 间距改善至 0.3m (log14)，但仍不够
- **0.3s:** 当前折中值

### 对比数据

| 限流值  | log         | vtx2-vtx3 间距 | skipped 帧数 |
|------|-------------|--------------|------------|
| 0.5s | log5/log12  | 0.7m         | ~83%       |
| 0.1s | log13       | 0.2m         | ~80%       |
| 0.2s | log14       | 0.3m         | ~65%       |
| 0.3s | log15/log16 | 0.44m        | ~50%       |

## 修复

[config/scout\\_rosbag.yaml:28](#):

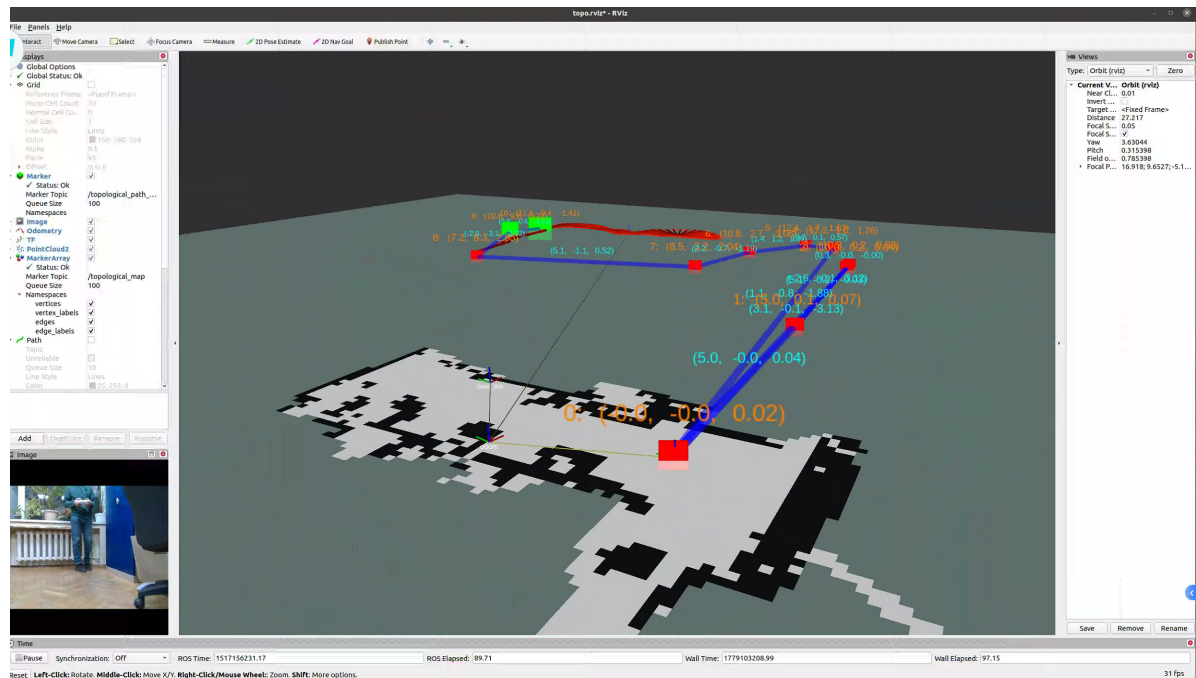
```
topomap:  
  pcd_process_interval: 0.3
```

[src/prism\\_topomap\\_node.cpp:84-85](#) 从 config 读取, [src/prism\\_topomap\\_node.cpp:499](#) 使用 `pcd_process_interval_` 替代硬编码 `0.5`。

```
THROTTLE] Skipping PCD frame (stamp=1517156231.040, diff=0.101s, skipped_total=655, queue=1, gt_buf=0, odom_buf=897)esc[0m
```

## Bug 5: `addNewVertex` 回环边无距离校验 → 幻影边 + 错误节点链

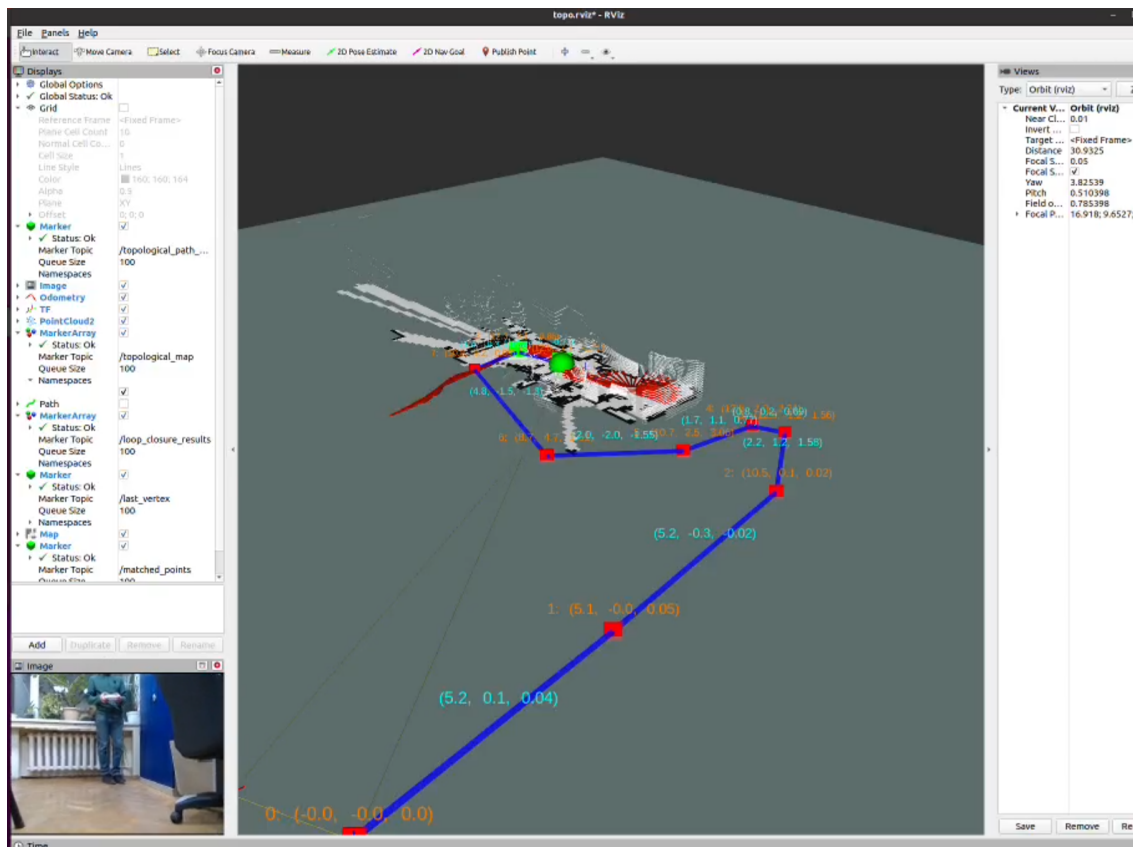
### 现象 (最严重的 bug)



log14 中 RVIZ 观察到:

- vertex 3 到 vertex 0 有一条蓝色边, 但实际不应直接相连
- vertex 4 到 vertex 0 也有一条边, 但初始位置和房间不可能直接到达 (长走廊拐角进房间位置)
- vertex 2 和 3 距离仅 0.3m

原始长这样, 不会有上图里面初始节点和拐角的多余蓝边



## 因果链分析

**第 1 步** — 定位配准模型产生高分但几何错误的 `re1_poses` :

```
log12: gridRegistration type=localization score=0.809 trans=(323.52,407.83,2.837 rad)
```

配准模型在网格不重叠时仍返回高置信度 ( $0.809 > 0.6$  阈值) , 但像素变换 (323, 407) 在  $360 \times 360$  栅格上处于边缘, 转到度量坐标后产生物理上不可能的相对位姿。

**第 2 步** — `addNewVertex` 无校验创建幻影边:

log14 line 537-540:

```
Add edge (10.3,0.2)->(5.0,0.1) re1_pose=(-2.6,-0.2,-3.12) ← 错误
Add edge (10.3,0.2)->(-0.0,-0.0) re1_pose=(3.1,-0.2,3.13) ← 大错特错
```

- vtx3 全局坐标 (10.34, 0.23), vtx0 全局坐标 (-0.04, 0.00), 实际距离 10.4m
- 边声称 `pred_dist=3.1m`, 差了  $3.4\times$
- Python 原版 [prism\\_topomap.py:413](#) 此处有注释掉的校验 `#if np.sqrt(...) < 5:`, C++一开始同样没加校验

**第 3 步** — `reattachByEdge` 沿幻影边错误跳转:

log14 line 676:

```
Edge reattach: from vertex 3 to vertex 0 (match_dist=0.03)
```

`reattachByEdge` 遍历 vertex 3 的所有邻居 (包括刚创建的错误边) , 发现 vertex 0 的边声称最近, scan matching "确认"后跳转到 vertex 0。

**第 4 步** — 从错误节点派生后续顶点:

log14 line 695-697:

```
Add edge (-0.0,-0.0)->(12.3,1.0) rel_pose=(1.12,-0.80,-1.88) dist=1.38
```

vertex 4 创建时 `last_vtx=0`，边连到 vertex 0。边声称距离 1.38m，但实际 12.3m。

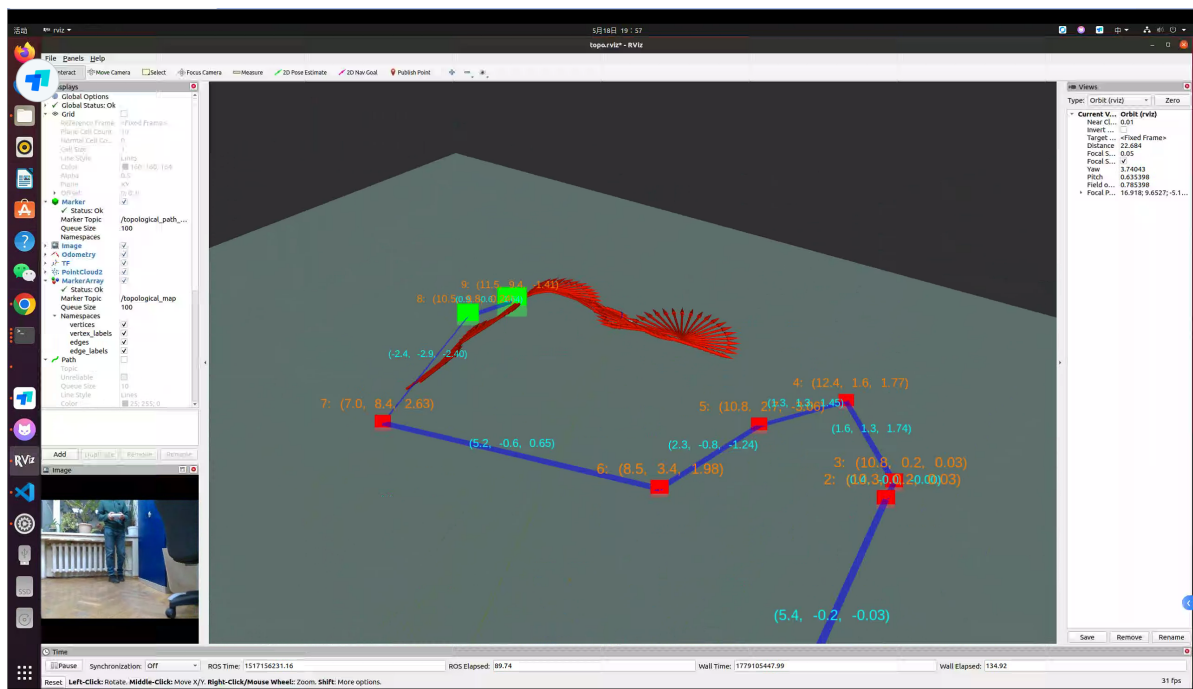
## 修复

[src/topo\\_slam\\_model.cpp:530-553](#) — 两层校验：

```
// 第 1 层: Python 原意 - 预测边长不超过 max_edge_length
double pred_dist = sqrt(pred_rel_pose[0]^2 + pred_rel_pose[1]^2);
if (pred_dist > max_edge_length_) { continue; }

// 第 2 层: 距离一致性 - 预测距离应与直接距离一致
double direct_dist = distance(new_vertex, matched_vertex);
double ratio = max(pred_dist, direct_dist) / min(pred_dist, direct_dist);
double abs_diff = abs(pred_dist - direct_dist);
if (abs_diff > 3.0 && ratio > 2.0) { continue; } // 差异 > 3m 且 > 2x
```

## 结果对比



| 指标                | 修复前 (log14)            | 第一次修复 (log15)      | 最终修复 (log16)          |
|-------------------|------------------------|--------------------|-----------------------|
| 错误边 3→0           | 创建                     | 创建 (公式太松)          | 拒绝 ratio=4.3 diff=8.3 |
| 错误边 3→1           | 创建                     | 创建 (公式太松)          | 拒绝 ratio=3.9 diff=4.3 |
| Edge reattach 3→0 | 触发 match_dist=0.03     | 触发 match_dist=0.07 | 未触发                   |
| vtx4→vtx0 幻影边     | 出现 (距离 1.4m, 实际 12.3m) | 出现                 | 未出现                   |
| 顶点数               | 11                     | 12                 | 9                     |
| 顶点顺序链             | 0→1→2→3→0(跳)→4→...     | 0→1→2→3→0(跳)→4→... | 0→1→2→3→4→5→6→7→8→9   |

此外这里的代码状态机处理流程跟论文不一样，如果说论文是1234的顺序，代码实现是2134，原始的和  
我重构的一样（沿边切换最优先）但是从效果上来说，最终可以等价

## 其他改动

### 可视化修复

[src/results\\_publisher.cpp:80-163](#) — C++ `publishGraph` 缺少边的相对位姿文字和顶点完整信息。修复后与 Python 一致：每条边中点显示 `(rel_x, rel_y, rel_theta)`，顶点显示 `"id: (x, y, theta)"`。

